Governors State University
OPUS Open Portal to University Scholarship

All Capstone Projects | Student Capstone Projects

Fall 2024

Quest: Navigating the Future of Communication

Musa Maali
Governors State University

Follow this and additional works at: https://opus.govst.edu/capstones

Recommended Citation
Maali, Musa, "Quest: Navigating the Future of Communication" (2024). *All Capstone Projects*. 694.
https://opus.govst.edu/capstones/694

For more information about the academic degree, extended learning, and certificate programs of Governors State University, go to http://www.govst.edu/Academics/Degree_Programs_and_Certifications/

Visit the Governors State Computer Science Department
This Capstone Project is brought to you for free and open access by the Student Capstone Projects at OPUS Open Portal to University Scholarship. It has been accepted for inclusion in All Capstone Projects by an authorized administrator of OPUS Open Portal to University Scholarship. For more information, please contact opus@govst.edu.

Quest: Navigating the Future of Communication

By

Musa Maali
B.S, Governors State University, 2021

GRADUATE CAPSTONE SEMINAR PROJECT

Submitted in partial fulfillment of the requirements

For the Degree of Master of Science,

With a Major in Computer Science

Governors State University
University Park, IL 60484

2024

ABSTRACT

Quest is a new innovative messaging web application designed to enhance and redefine the way users communicate, connect, and collaborate. Drawing inspiration from successful platforms such as Discord and Telegram, Quest's goal is to provide a seamless and immersive experience for users seeking a dynamic and feature – rich collaboration environment. The core philosophy of Quest is to create a platform that facilitates real time communication and shared interests and goals. The app will include a user-friendly interface, ensuring easy access, navigation, and accessibility for users of all different backgrounds. Key focuses that this web application will include be security authorization of user accounts, sending back and forth messages to different users across the messaging platform and lastly the user will be able to customize a user profile of their own.

Table of Content

1 Project Description 1
1.1 Competitive information 1
1.2 Future Enhancements 1
1.3 Definitions and Acronyms 1
2 Project Technical Description 2
2.1 Application Information flows 3
2.2 Diagrams & Interfaces 4
2.3 Interactions with other Applications 7
2.4 Capabilities 7
2.5 Risk Assessment and Management 7
3 Project Requirements 8
3.1 Identification of Requirements 8
3.2 Operations, Administration, Maintenance and Provisioning (OAM&P) 9
3.3 Security and Fraud Prevention 9
3.4 Release and Transition Plan 9
4 Project Design Description 9
5 Internal/external Interface Impacts and Specification 13
6 Design Units Impacts 13
6.1 Functional Area A/Design Unit A: User Authentication 14
6.1.1 Functional Overview 14
6.1.2 Impacts 14
6.1.3 Non-Functional Requirements 14
7 Appendix 15
7.1 Front-end code examples 15
7.2 Back-end code examples 16
8 References 19

1 Project Description

Quest is an innovative messaging platform that redefines real-time communication by combining advanced technology with a community focused design. Unlike traditional messaging applications that prioritize connecting users with their existing contacts, Quest empowers users to engage and discover new people, fostering meaningful interactions. The platform offers a secure environment with end-to-end encryption and robust authorization protocols to protect user accounts and communications. Its real-time messaging capabilities enable seamless back and forth communication, supported by low latency technology for a quick responsive user experience. This is what allows Quest to deliver an unparalleled experience that encourages connection and engagement like never before

1.1 Competitive information

In a market dominated by platforms like Facebook Messenger and Instagram, which focus on broad social networking and messaging, Quest differentiates itself by emphasizing the discovery of the right communities. Unlike mainstream apps that connect users primarily with people they already know, Quest helps users find and engage with new and different people across the world which is different than connecting with contacts you may already know. This targeted approach fosters more meaningful connections and collaborations.

1.2 Future Enhancements

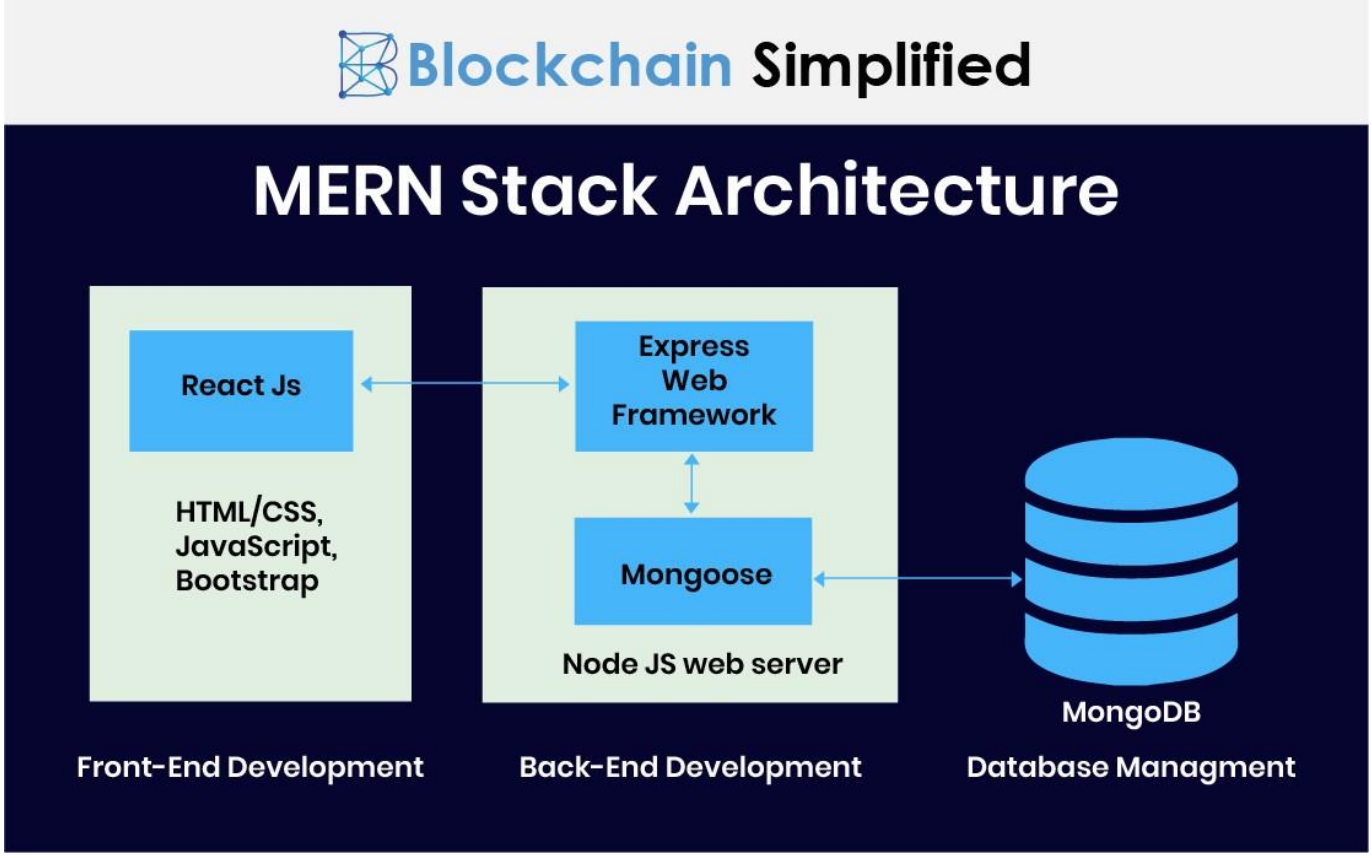
Looking ahead, Quest plans to introduce several key features to enhance the user experience and expand the platform's capabilities. One of the primary enhancements is the addition of group messaging. This feature will allow users to create group chats where multiple participants can communicate simultaneously within the same conversation. Group messaging will facilitate better collaboration among teams, social circles, and communities, making Quest an even more versatile tool for both personal and professional communication. Another significant enhancement is the integration of AI-driven messaging. By incorporating artificial intelligence, Quest aims to offer intelligent features such as predictive text suggestions, smart replies, and personalized message recommendations. AI can also assist in improving security by detecting spam or suspicious activities and providing users with options to filter unwanted messages. These advancements will streamline communication and make interactions more efficient and tailored to individual user preferences. These future enhancements are designed to make Quest a more comprehensive and user-friendly platform. By continually evolving and adding new features, Quest aims to meet the changing needs of its users and maintain a competitive edge in the rapidly developing landscape of digital communication.

1.3 Definitions and Acronyms

- HTML – Hyper Text Markup Language
- CSS – Cascading Style Sheets
- MFA – Multi Factor Authorization
- JWT: JSON Web Token
- CORS: Cross-Origin Resource Sharing

2 Project Technical Description

The architecture of Quest is built upon a modern and robust technology stack designed to deliver a responsive, secure, and feature-rich messaging platform. The application consists of a React.js front-end, a Node.js/Express.js back-end server, and MongoDB for database management. On the front-end, Quest utilizes React.js, a powerful JavaScript library for building user interfaces. Reacts component-based architecture allows developers to create reusable UI components, leading to efficient development and maintainable code. This modular approach enhances the scalability of the application and makes it easier to implement new features or modify existing ones. To style the user interface, Quest leverages Tailwind CSS, a utility-first CSS framework. Tailwind provides a wide range of pre-defined classes that simplify the styling process and ensure a consistent design throughout the application. By using Tailwind, developers can quickly build complex layouts and customize components without writing extensive CSS from scratch. This results in a dynamic and visually appealing user interface that enhances the overall user experience. At the back end, Quest employs Node.js as the runtime environment, with Express.js serving as the web application framework. Node.js allows for scalable server-side scripting using JavaScript, which streamlines development by enabling the use of a single language on both the front-end and back-end. Express.js simplifies the creation of web servers and APIs, providing essential features like routing, middleware support, and request handling. For database management, Quest uses MongoDB, a NoSQL database known for its scalability and flexibility in handling unstructured data. MongoDB stores data in JSON-like documents, which aligns well with the JavaScript-centric technologies used throughout the application. The database stores all essential data, including user profiles, authentication details, messages, and any other relevant information. To handle communication between the front-end and back-end, especially when they are hosted on different domains or ports, Quest configures Cross-Origin Resource Sharing (CORS). Proper CORS setup ensures that the application adheres to web security standards while allowing legitimate requests to pass through. In summary, Quest's technical architecture is a well-integrated combination of front-end and back-end technologies that work together to provide a secure, efficient, and user-friendly messaging platform. The use of React.js and Tailwind CSS on the front-end results in a dynamic and modern user interface. Meanwhile, Node.js and Express.js on the back end handle the server-side logic, real-time communication, and data management with MongoDB. This architecture not only meets the current requirements of the application but also lays a solid foundation for future enhancements and scalability.

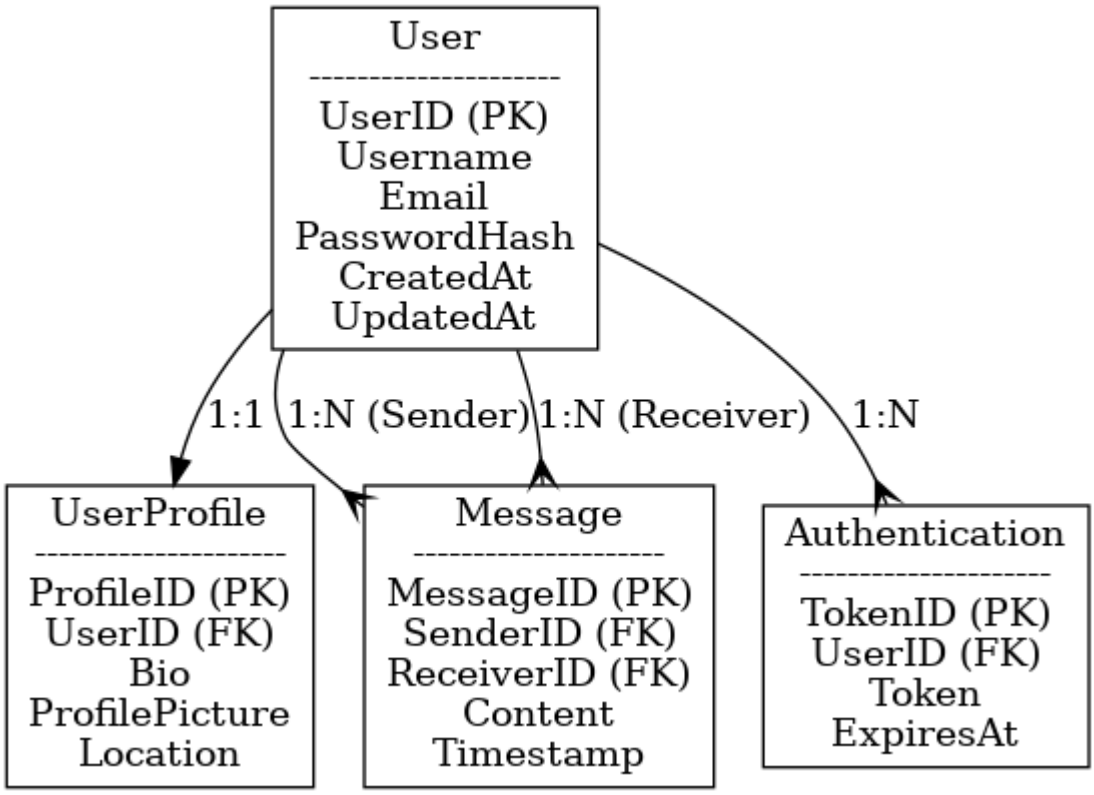


Simplified explanation of the MERN stack that Quest uses.

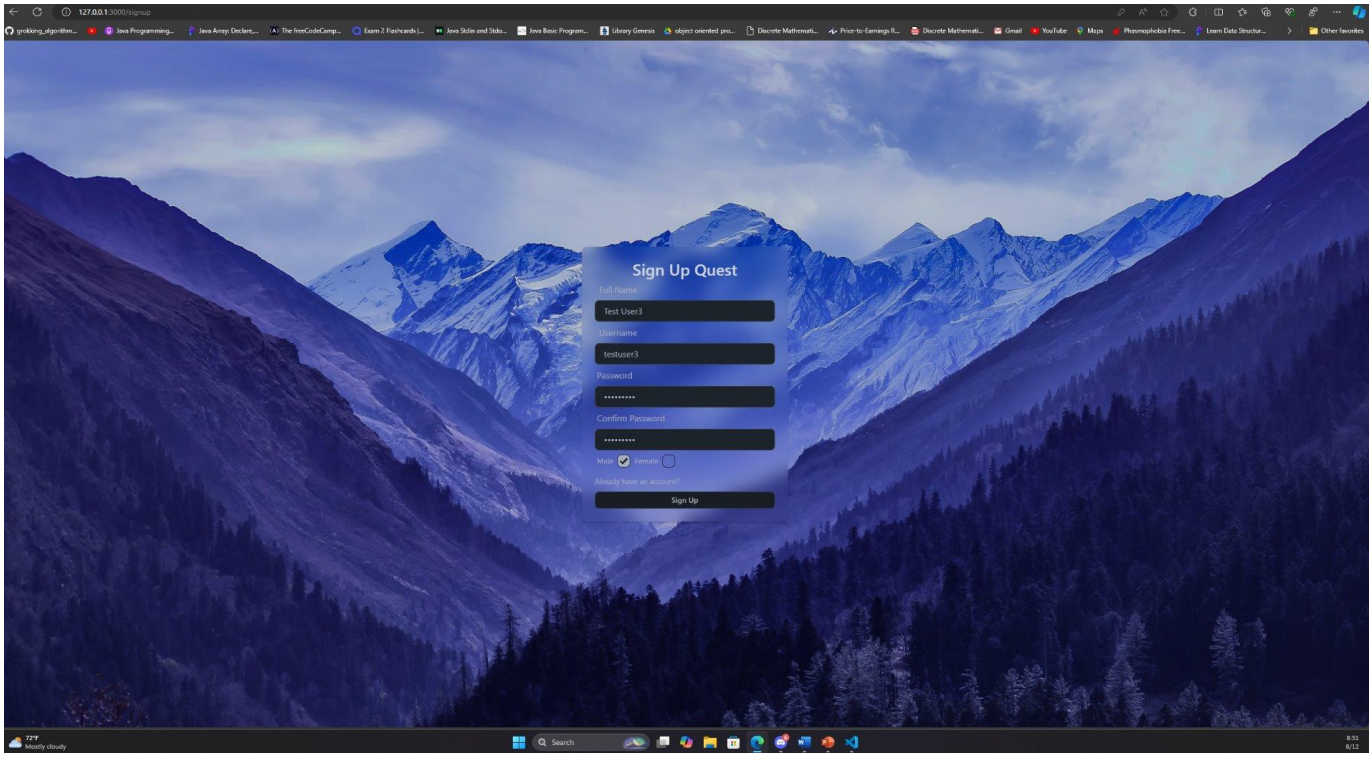
2.1 Application Information flows

1. User Registration/Login
2. User Interaction:
3. Message Threads
4. Profile Picture Images
5. Searching For Users
6. Exploring user profiles
7. Notification management
8. Signing Up
9. Logging Out

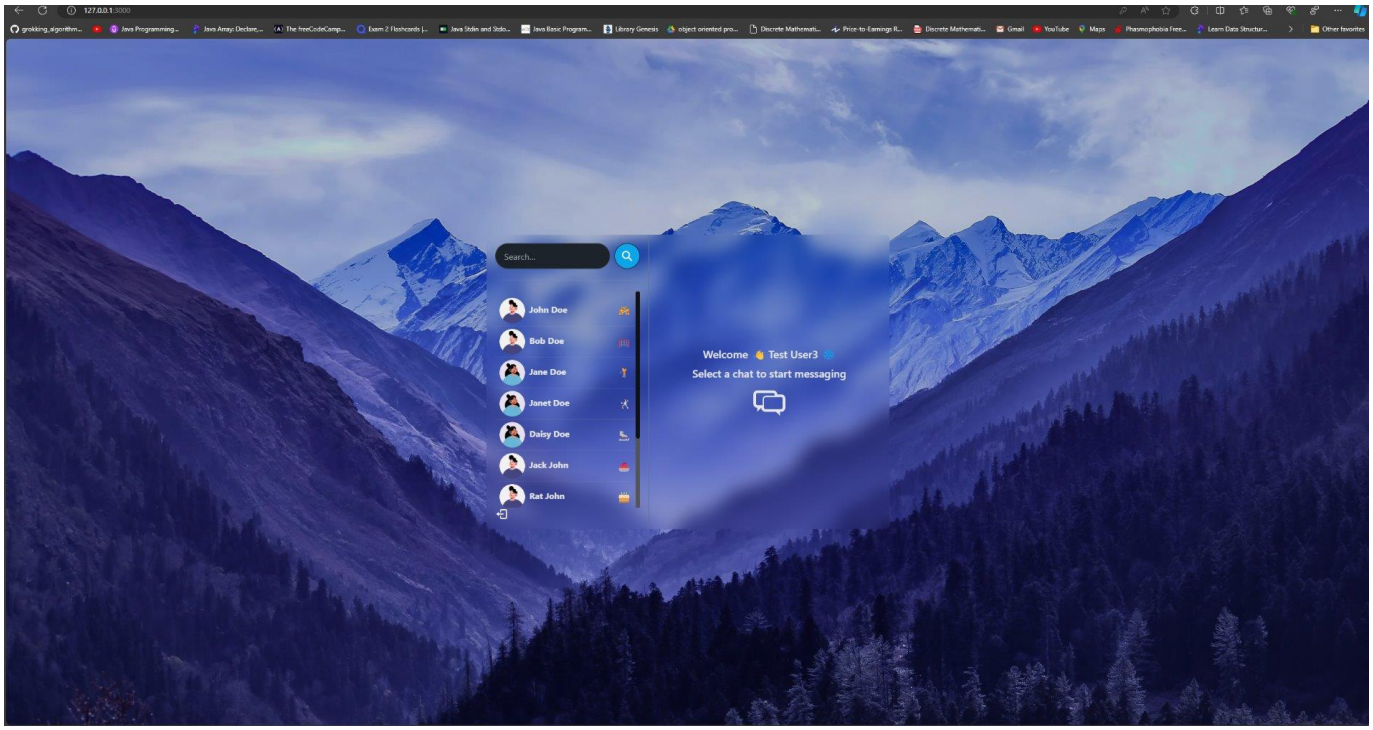
2.2 Diagrams & Interfaces



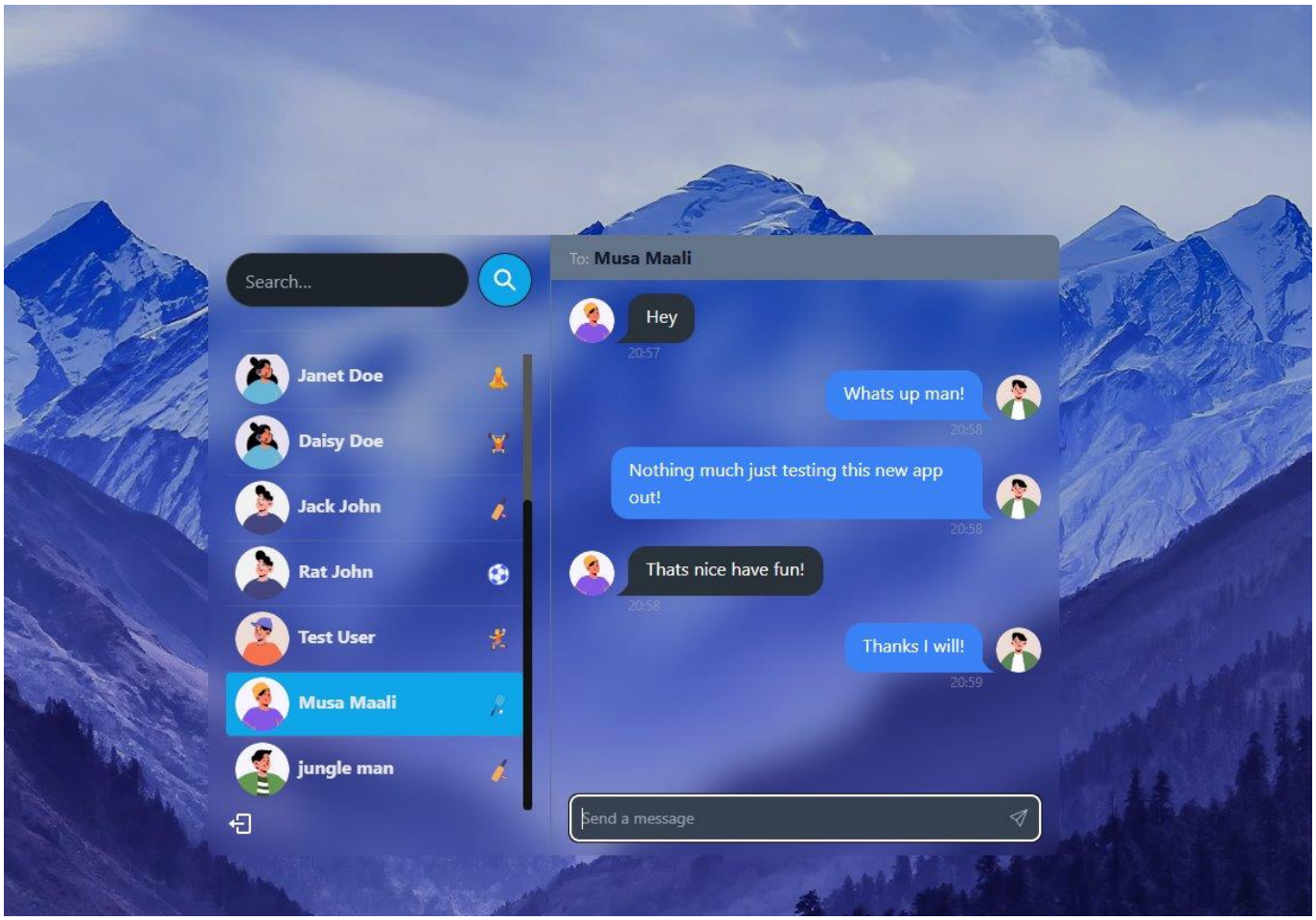
Entity Relationship Diagram (ER)



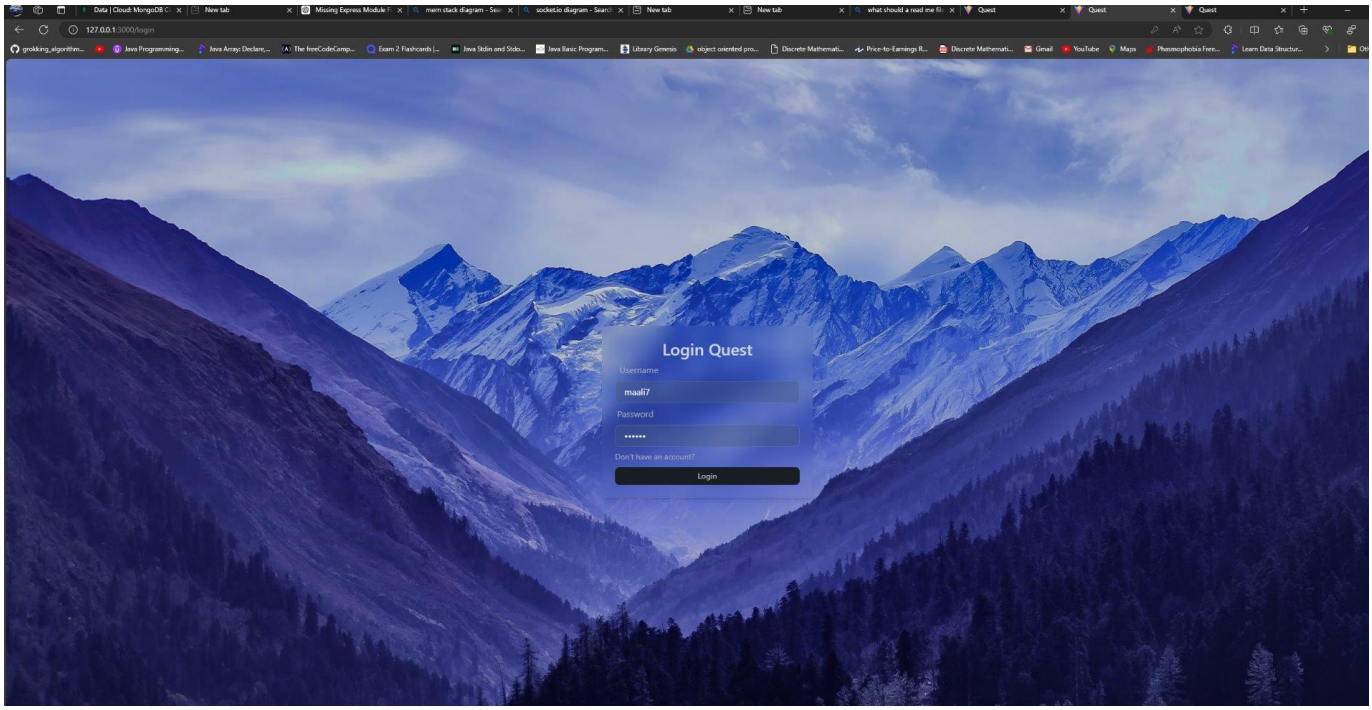
Sign-up Page



Home Page

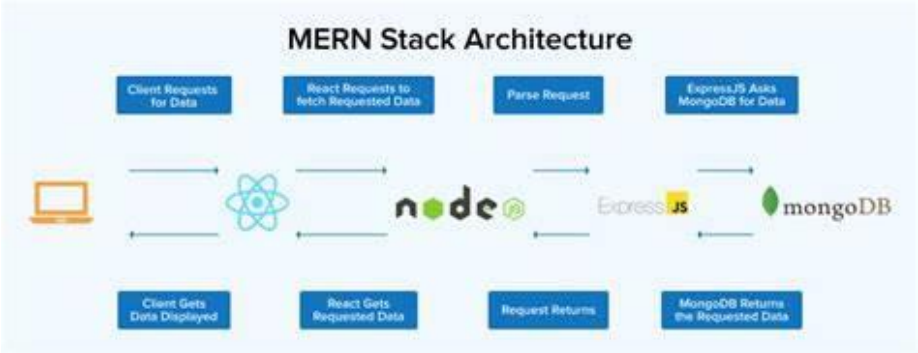


Chat Example #1



Login Page

Chat Example #2



<u>The above picture is an example of the MERN stack architecture incorporated into Quest.</u>

2.3 Interactions with other Applications

Future updates of Quest may include integrations with platforms like slack or teams allowing for cross-platform messaging.

2.4 Capabilities

1. ***User Authentication and Authorization***
   - *Secure user login with JWT and cookie-based server sessions*
2. ***Real-Time Messaging***
   - *Facilitate real-time bidirectional communication between users and the server.*
   - *Instant message delivery handling using WebSocket's*
3. ***Channel Management***
   - *Create, join, and manage chats with other users.*
4. ***Profile Customization***
   - *Ability to enable users to update their profiles with avatars for male/female genders.*
5. ***Notification Management***
   - *Capability to hear notifications go off via ringtone when a message is received.*
6. ***Database Operations***
   - *Robust data handling through MongoDB, ensuring scalability and data integrity.*
7. ***Scalability & Performance***
   - *Ability to scale the application to accommodate future growth and a growing user base.*
   - *Capability to ensure efficient user resource utilization for good overall performance*.

2.5 Risk Assessment and Management

1. ***Security Vulnerabilities***
   - Risk: Data breaches, unauthorized access, or potential injection attacks
   - Resolution: Implementing security measures such as encryption, authentication mechanisms, and performing regular audits to address potential security risks
2. ***Technical challenges***
   - Risk: Understanding the technology and code behind the architecture stack and allowing the program to run successfully
   - Resolution: Learning the stack and increasing the quality of testing to allow for a smooth-running application

3. Data Loss

- Risk: Data loss due to system failures, hardware failures, and human errors can potentially result in loss of critical user information and disrupt user experiences
- Resolution: Implementing a data backup mechanism to ensure no data loss and utilizing version control systems such as git to track revisions and changes

3 Project Requirements

3.1 Identification of Requirements

<Quest-SP2024-1 User-Capability-000100>

Quest should allow users to send messages to individual users or channels.

- *Implementation: Mandatory*

<Quest-SP2024-1 User-Capability-000200>

Quest should support real-time updates for message delivery and notifications.

- *Implementation: Mandatory*

<Quest-SP2024-1 User-Capability-000300>

Quest must allow current users to create avatars for their own profiles.

- *Implementation: Mandatory*

<Quest-SP2024-1 User-Capability-000400>

The application should provide users with the ability to search for users across the platform.

- *Implementation: Mandatory*

<Quest-SP2024-1 User-Capability-000500>

Quest must allow users to register, log in, and authenticate securely using JWTs

- *Implementation: Mandatory*

<Quest-SP2024-1 User-Capability-000600>

The application must allow the users to manage messaging history with other users.

- *Implementation: Mandatory*

<Quest-SP2024-1 User-Capability-000700>

Quest should support emoji reactions for messages, allowing users to react to messages with a range of different emotions.

- *Implementation: Mandatory*

<Quest-SP2024-1 User-Capability-000800>

Application must allow users to select gender upon registration.

- *Implementation: Mandatory*

<Quest-SP2024-1 User-Capability-000900>

Quest must offer state-of-the-art security measures, including data encryption, secure authentication, and regular security audits.

- *Implementation: Mandatory*

3.2 Operations, Administration, Maintenance and Provisioning (OAM&P)

Including capabilities like data encryption, and security auditing will safeguard user data and prevent unauthorized access.

3.3 Security and Fraud Prevention

Quest's security and fraud prevention strategies are critical to maintaining the trust and safety of its user base. The platform employs multiple layers of security measures, including:

- **Data Encryption Algorithms**: Quest utilizes industry-standard encryption algorithms, such as Advanced Encryption Standard (AES) for data at rest and Transport Layer Security (TLS) for data in transit. These encryption methods ensure that user data remains confidential and protected from interception and unauthorized access.
- **User Authentication**: A secure authentication process is in place, incorporating the use of JSON Web Tokens (JWTs) for session management. Multi-Factor Authentication (MFA) is optionally implemented to provide an additional layer of security, requiring users to verify their identity through a second factor, such as a mobile app or SMS code.

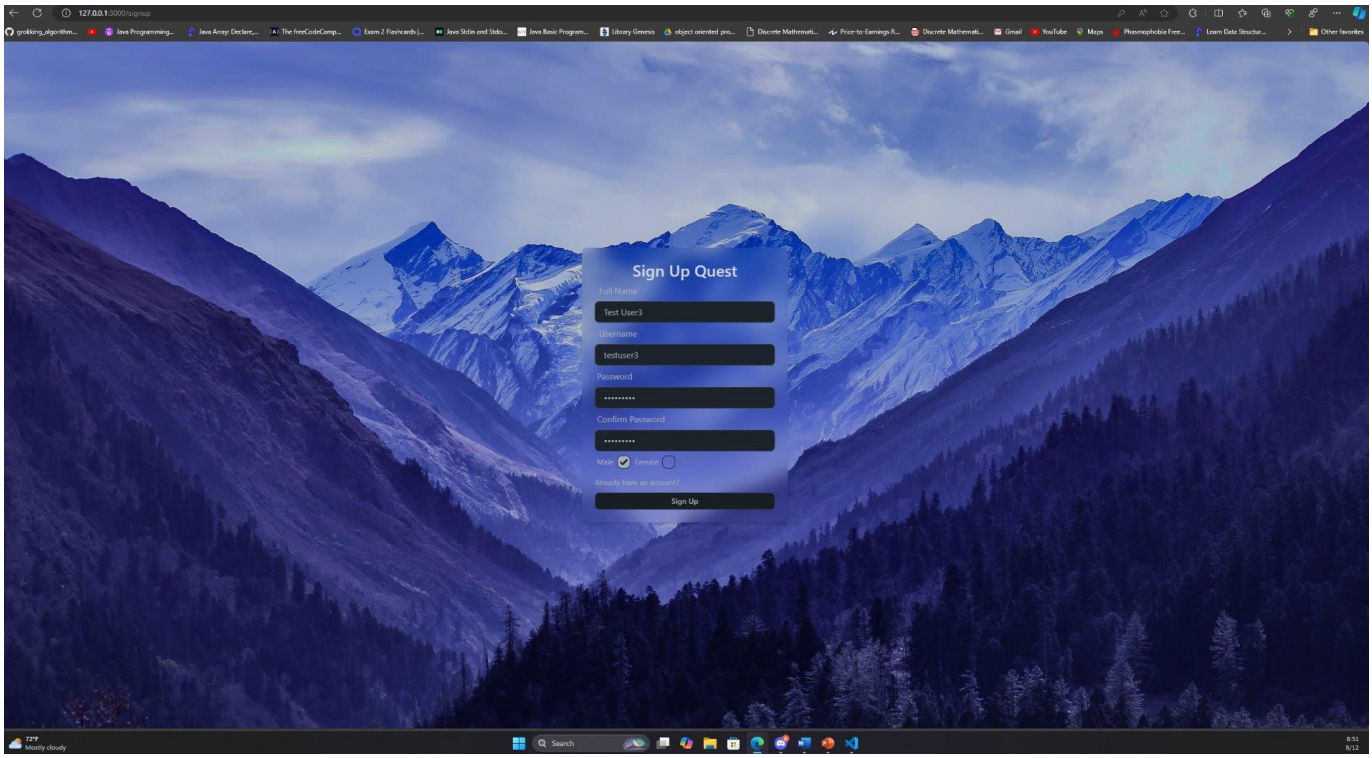
.

3.4 Release and Transition Plan

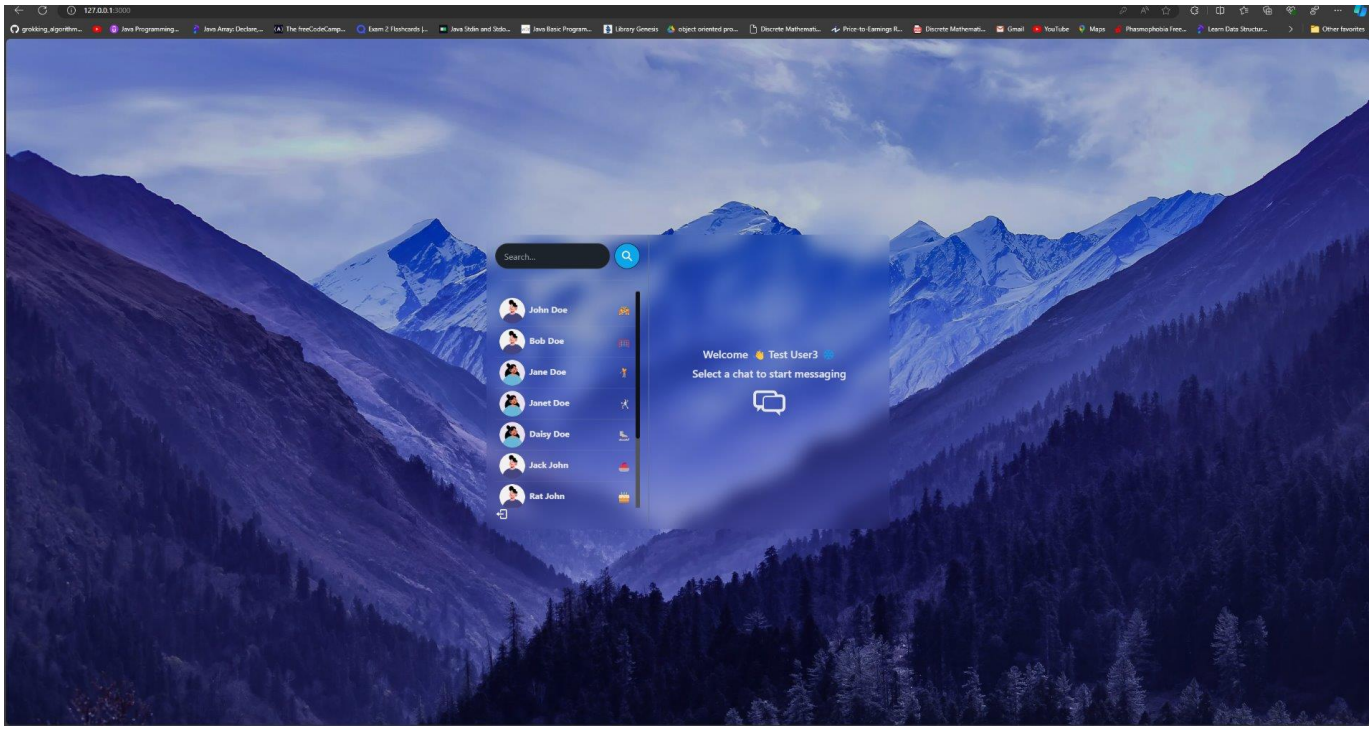
Deployment of application will be executed using close monitoring to address and introductory issues Quest may have and an option to leave feedback on ways Quest can be improved post release with an additional update.

4 Project Design Description

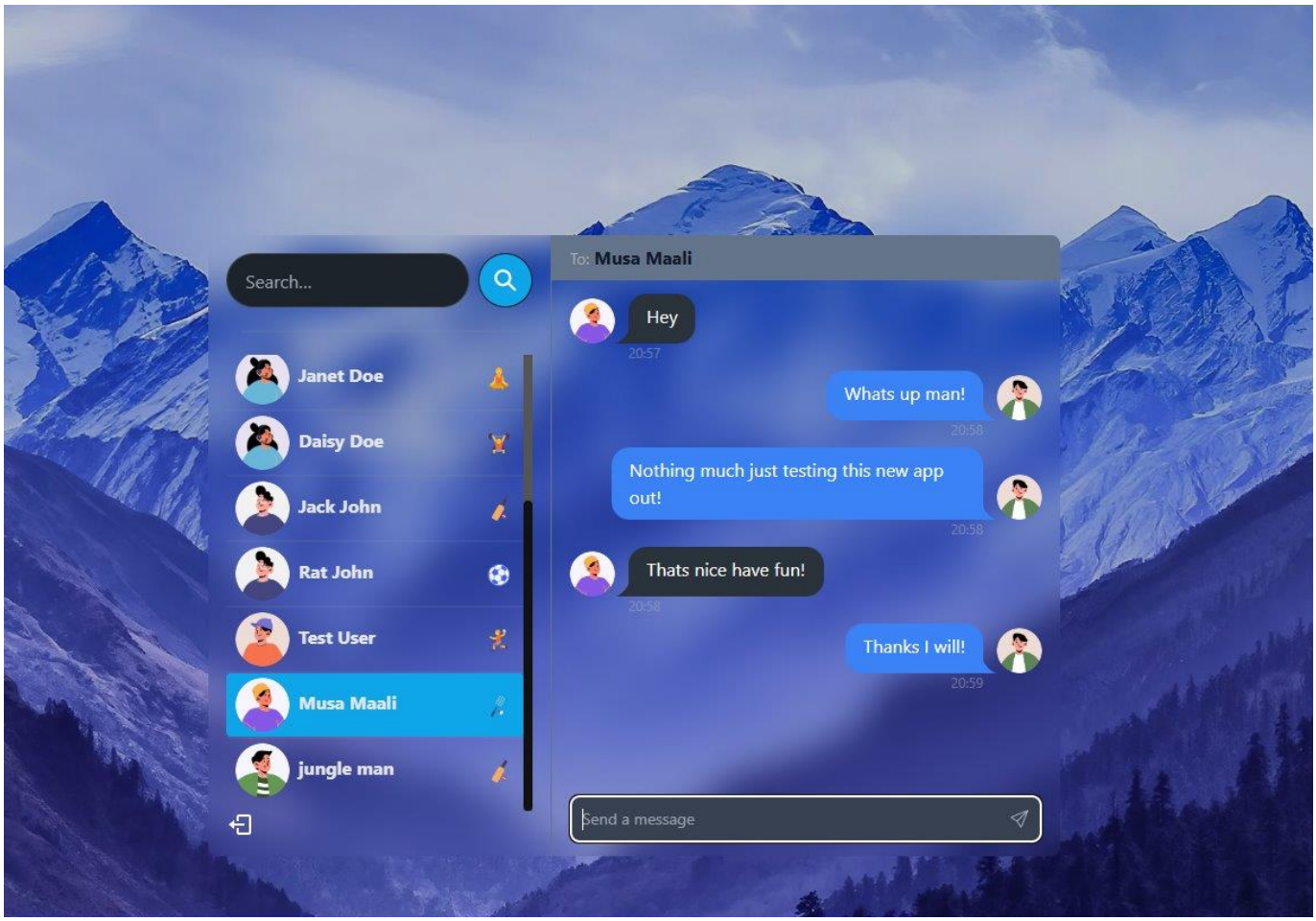
Quest's detailed project design introduced several interconnected components working side by side seamlessly to provide users of the application with a new and exciting platform. At the front end React.js organizes user interface components in a hierarchy, offering reusability, while CSS Tailwind handles the styling for a relaxed interface. Node.js and Express manage server-side operations. Data flows are achieved by HTTP, WebSocket, and RESTful API's, enabling smooth communication between clients.



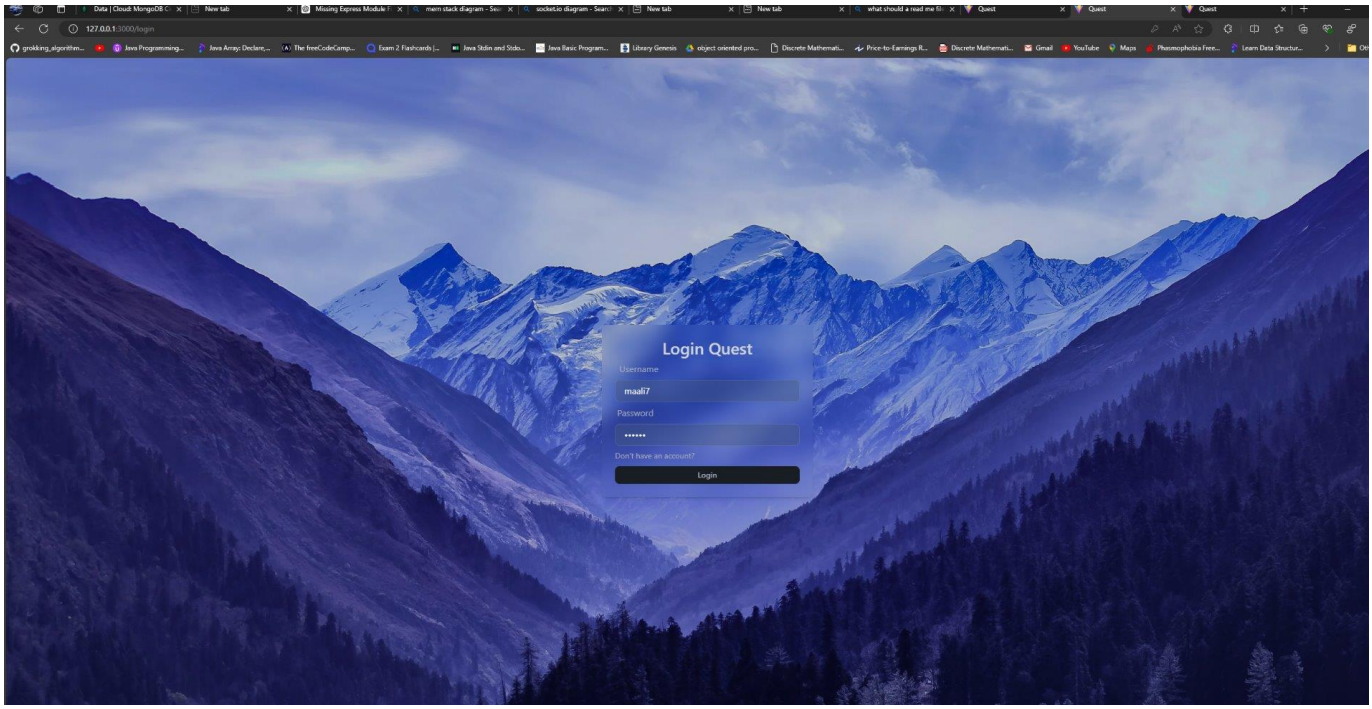
Signing up to Quest



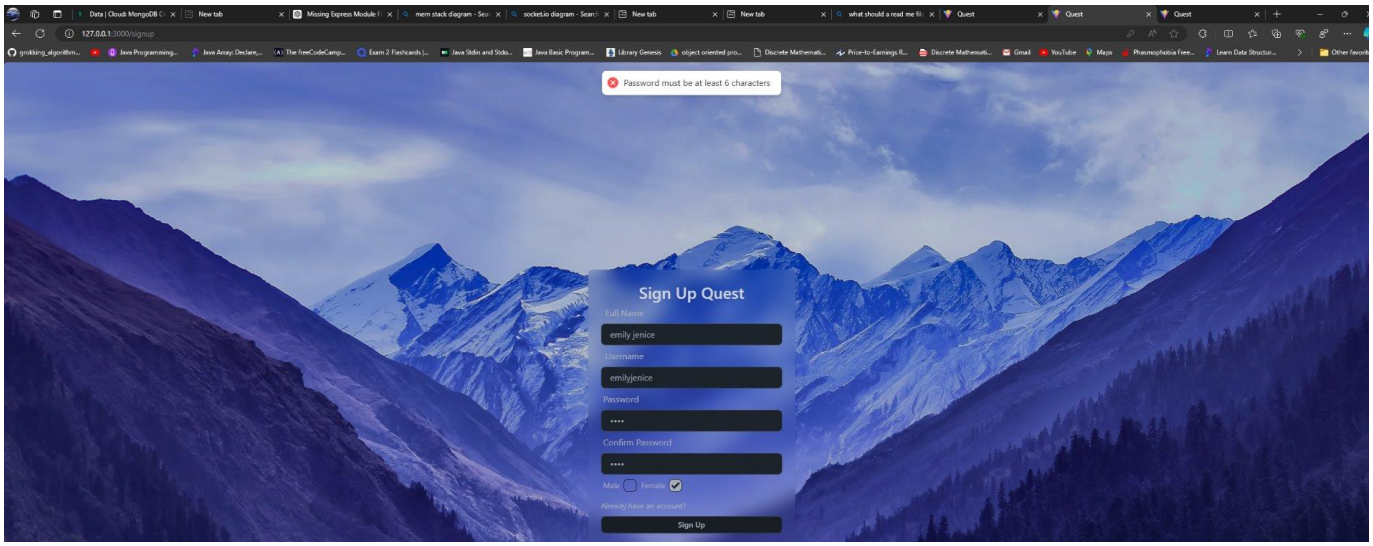
Welcome page of Quest with a list of all your possible contacts and chats



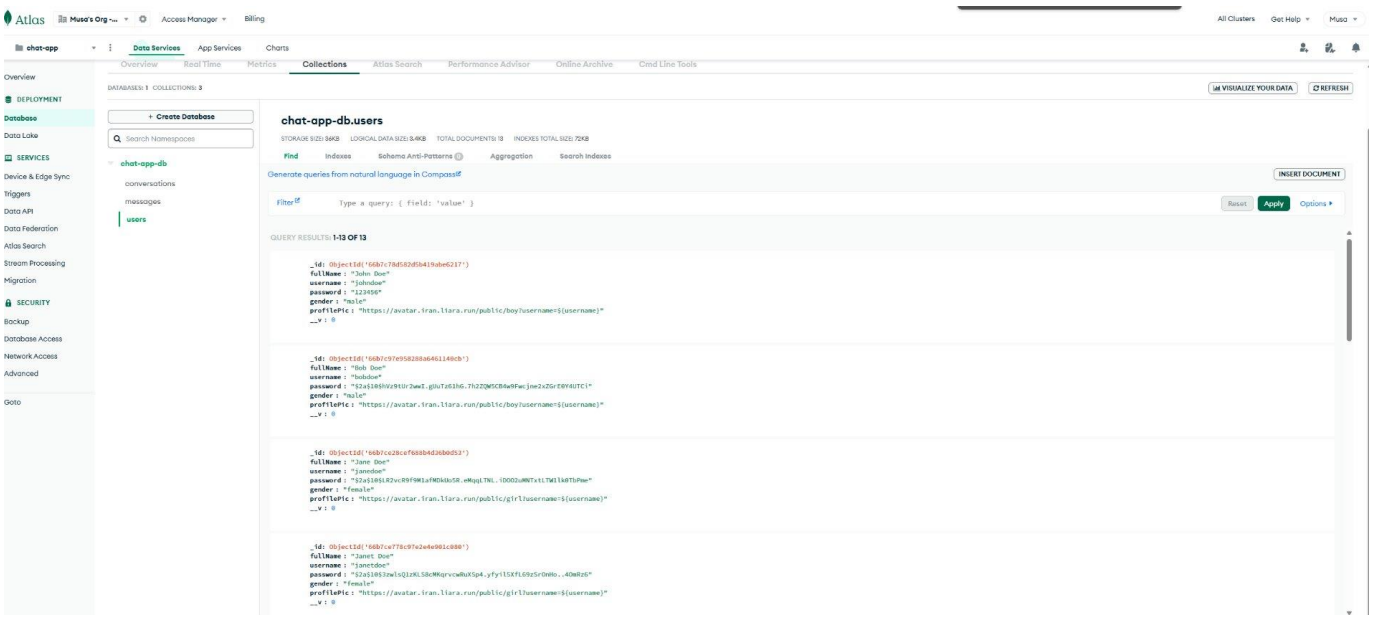
Here the test user that we created is interacting with my first user account



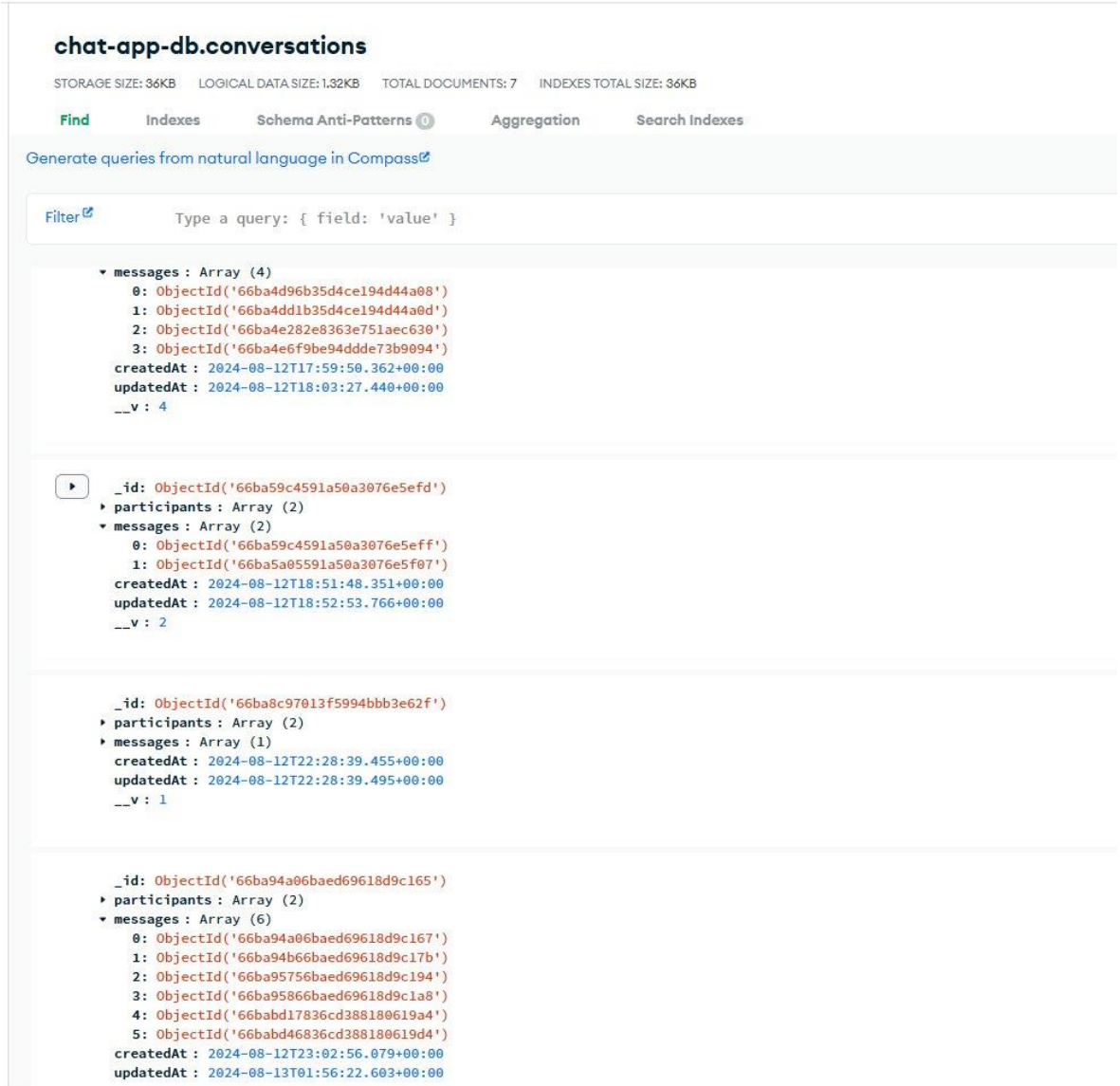
Login page after signing up and creating an account



Showing the error handling notification that a user will receive if the password is weak.



MongoDB with each object user including details such full name, username, password, gender, and profile picture.



Conversation objects that are added into the DB anytime a message is sent or received.

5 Internal/external Interface Impacts and Specification

- Third-Party Integrations:
- Possible integration with cloud storage providers such as Microsoft Azure or Google drive
- API Documentation Updates:
- Any changes to the API endpoints, including new features or modifications to existing ones, will require updates to the API documentation. Updates to reflect changes in request/response formats.
- Database Changes:
- As quest evolves, the database schema may need to be updated to support new data entities or relationships.

6 Design Units Impacts

1. **Quest Messaging Application:**

- *Impacts*:
    - **Frontend:** Will require updates to UI components to include new features overtime
    - **Backend:** Necessary modifications to API endpoints, WebSocket communications, and a database schema that pertains to Quests' requirements if time permits *

2. **Database Management:**
    - *Impacts*:
        - ***Schema Optimization:*** Regular optimization of the quest database scheme to accommodate random increases of users.
        - ***Backup and Recovery:*** Introducing more enhanced security measures such as encryption and access control to potentially protect user data from any phishing attempts.

3. **Third-Party integrations:**
    - *Impacts*:
        - **Dependency Management:** Potential updates may be required to interface configurations to ensure a smooth experience.

6.1 Functional Area A/Design Unit A: User Authentication

The design units within Quest encompass various areas, each with its specific impacts and requirements. As quest continues to evolve, these units will undergo reoccurring updates to incorporate new features, enhance performance, and to ensure the security and privacy of our user base.

6.1.1 Functional Overview

User Authentication provides users with a sense of security and safety.

6.1.2 Impacts

When a user is first registering for an account, we require them to confirm the password they're inputting twice to make sure that the password is something the user will remember. When the user creates a password, it undergoes a hash algorithm so that its safe and secure

6.1.3 Non-Functional Requirements

- **<Quest-SP2024-1 Scalability -000100>**

  Quest must be scalable to handle increasing numbers of users and messages without performance degrading.
    - *Implementation: Mandatory*

- **<Quest-SP2024-1 Security-000200>**

Quest must implement robust security measures, including data encryption and secure authentication

- *Implementation: Mandatory*

- **<Quest-SP2024-1 Performance-000300>**
  The application should provide a smooth and responsive user experience to keep a consistent user base.
  - Implementation: Mandatory

- **<Quest-SP2024-1 Reliability-000400>**
  Quest must ensure data integrity and reliability, with backups in place to prevent data loss occurring.
  - *Implementation: Mandatory*

7 Appendix

7.1 Front-end code examples

Login.jsx U

frontend > src > pages > login > Login.jsx > Login

```
import { useState } from "react";
import { Link } from "react-router-dom";
import useLogin from "../../hooks/useLogin";

const Login = () => {
    const [username, setUsername] = useState("");
    const [password, setPassword] = useState("");

    const { loading, login } = useLogin();

    const handleSubmit = async (e) => {
        e.preventDefault();
        await login(username, password);
    };
```

Login.js file renders the login page for the quest application. This is done by managing the state of the password and username and handling the form submission to authenticate users.

AuthContext.jsx U

frontend > src > context > AuthContext.jsx > ...

```
import { createContext, useContext, useState } from "react";

export const AuthContext = createContext();

// eslint-disable-next-line react-refresh/only-export-components
export const useAuthContext = () => {
    return useContext(AuthContext);
};

export const AuthContextProvider = ({ children }) => {
    const [authUser, setAuthUser] = useState(JSON.parse(localStorage.getItem("chat-user")) || null);

    return <AuthContext.Provider value={{ authUser, setAuthUser }}>{children}</AuthContext.Provider>;
};
```

The provided screen shot shows the creation of AuthContext and AuthContextProvder, which enables the entire app to access the authenticated user's current state.

```jsx
const SocketContext = createContext();

export const useSocketContext = () => {
    return useContext(SocketContext);
};

export const SocketContextProvider = ({ children }) => {
    const [socket, setSocket] = useState(null);
    const [onlineUsers, setOnlineUsers] = useState([]);
    const { authUser } = useAuthContext();

    useEffect(() => {
        if (authUser) {
            const socket = io("https://chat-app-yt.onrender.com", {
                query: {
                    userId: authUser._id,
                },
            });

            setSocket(socket);

            // socket.on() is used to listen to the events. can be used both on client and server side
            socket.on("getOnlineUsers", (users) => {
                setOnlineUsers(users);
            });

            return () => socket.close();
        } else {
            if (socket) {
                socket.close();
                setSocket(null);
            }
        }
    }, [authUser]);

    return <SocketContext.Provider value={{ socket, onlineUsers }}>{children}</SocketContext.Provider>;
};
```

The SocketContext.jsx file sets up a context for real time communication in Quest. When a user is logging in, it connects to a socket.IO server to enable real time features.

```
frontend > src > hooks > JS useListenMessages.js > default
import { useEffect } from "react";

import { useSocketContext } from "../context/SocketContext";
import useConversation from "../zustand/useConversation";

import notificationSound from "../assets/sounds/notification.mp3";

const useListenMessages = () => {
    const { socket } = useSocketContext();
    const { messages, setMessages } = useConversation();

    useEffect(() => {
        socket?.on("newMessage", (newMessage) => {
            newMessage.shouldShake = true;
            const sound = new Audio(notificationSound);
            sound.play();
            setMessages([...messages, newMessage]);
        });

        return () => socket?.off("newMessage");
    }, [socket, setMessages, messages]);
};
export default useListenMessages;
```

The useListenMessages.js file actively allows for listening of new chat messages within Quest.

7.2 Back-end code examples

```
JS connectToMongoDB.js ×
backend > db > JS connectToMongoDB.js > default
import mongoose from 'mongoose';

const connectToMongoDB = async () => {
    try {
        await mongoose.connect(process.env.MONGO_DB_URI);
        console.log("Connected to MongoDB");
    } catch (error) {
        console.log("Error connectingf to MongoDB", error.message);
    }
};

export default connectToMongoDB;
```

The fundamental connection of the MongoDB in Quest. This connects Quest to its DB

```
const server = http.createServer(app);
const io = new Server(server, {
    cors: {
        origin: ["http://localhost:3000"],
        methods: ["GET", "POST"],
    },
});

export const getReceiverSocketId = (receiverId) => {
    return userSocketMap[receiverId];
};

const userSocketMap = {}; // {userId: socketId}

io.on("connection", (socket) => {
    console.log("a user connected", socket.id);

    const userId = socket.handshake.query.userId;
    if (userId != "undefined") userSocketMap[userId] = socket.id;

    // io.emit() is used to send events to all the connected clients
    io.emit("getOnlineUsers", Object.keys(userSocketMap));

    // socket.on() is used to listen to the events. can be used both on client and server side
    socket.on("disconnect", () => {
        console.log("user disconnected", socket.id);
        delete userSocketMap[userId];
        io.emit("getOnlineUsers", Object.keys(userSocketMap));
    });
});
```

The Socket.js file allows for handling of real-time communications by Ques also keeps track of all connected users by mapping the socketIDs to the user IDs

```
JS generateToken.js ×
backend > utils > JS generateToken.js > default
import jwt from "jsonwebtoken";

const generateTokenAndSetCookie = (userId, res) => {
    const token = jwt.sign({ userId }, process.env.JWT_SECRET, {
        expiresIn: "15d",
    });


    res.cookie("jwt", token, {
        maxAge: 15 * 24 * 60 * 60 * 1000, // MS
        httpOnly: true, // prevent XSS attacks cross-site scripting attacks
        sameSite: "strict", // CSRF attacks cross-site request forgery attacks
        secure: process.env.NODE_ENV !== "development",
    });
};

export default generateTokenAndSetCookie;
```

The included functions create a JSOON Web Token for a userId and allows it to set it as a secure cookie in the response from HHTP. The inclusion of this cookie gives security attributes to protect against bad actors.

```
JS user.controller.js M ×
backend > controllers > JS user.controller.js > ...
import User from "../models/user.model.js";

export const getUsersForSidebar = async (req, res) => {
    try {
        const loggedInUserId = req.user._id;

        const filteredUsers = await User.find({ _id: { $ne: loggedInUserId } }).select("-password");

        res.status(200).json(filteredUsers);
    } catch (error) {
        console.error("Error in getUsersForSidebar: ", error.message);
        res.status(500).json({ error: "Internal server error" });
    }
};
```

The above function users.controllers.js retrieves a list of all users with the exception of the user that is currently logged in.

```
export const logout = (req, res) => {
    try {
        res.cookie("jwt", "", { maxAge: 0 });
        res.status(200).json({ message: "Logged out successfully" });
    } catch (error) {
        console.log("Error in logout controller", error.message);
        res.status(500).json({ error: "Internal Server Error" });
    }
```

The logout function in the auth.controllers.js file logs out a user's by clearing the existing jwt cookie by setting it to an empty string to expire immediately.

8 References

GfG. (2024, February 14). React tutorial. GeeksforGeeks.

https://www.geeksforgeeks.org/react-tutorial/

Node.Js tutorial. (n.d.). https://www.w3schools.com/nodejs/

GfG. (2024b, March 23). JavaScript tutorial. GeeksforGeeks.

https://www.geeksforgeeks.org/javascript/

Node.Js tutorial. (n.d.). https://www.w3schools.com/nodejs/